

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen

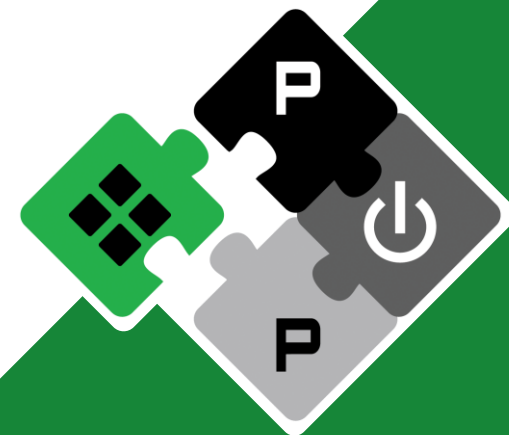
zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli
Luca Benini

avanelli@iis.ee.ethz.ch
lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform



pulp-platform.org



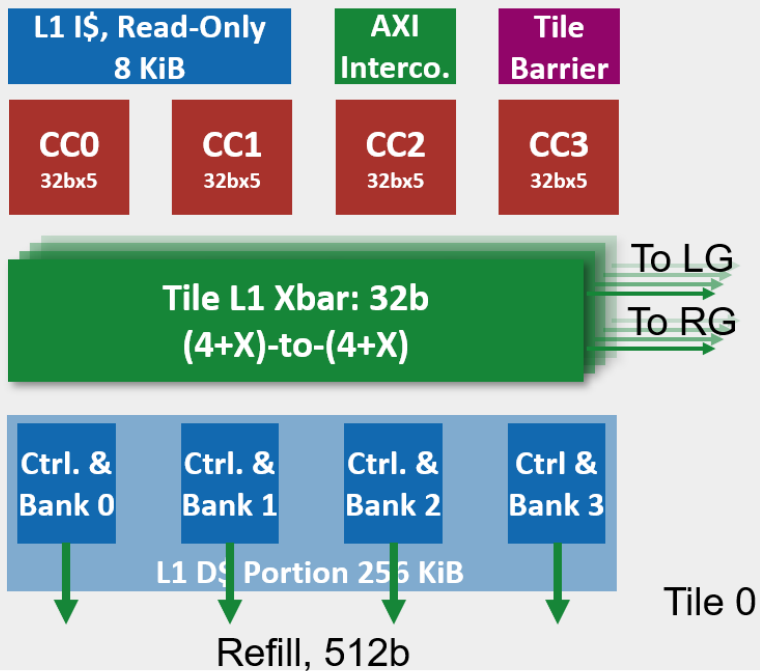
youtube.com/pulp_platform



Hardware Development (Cluster)



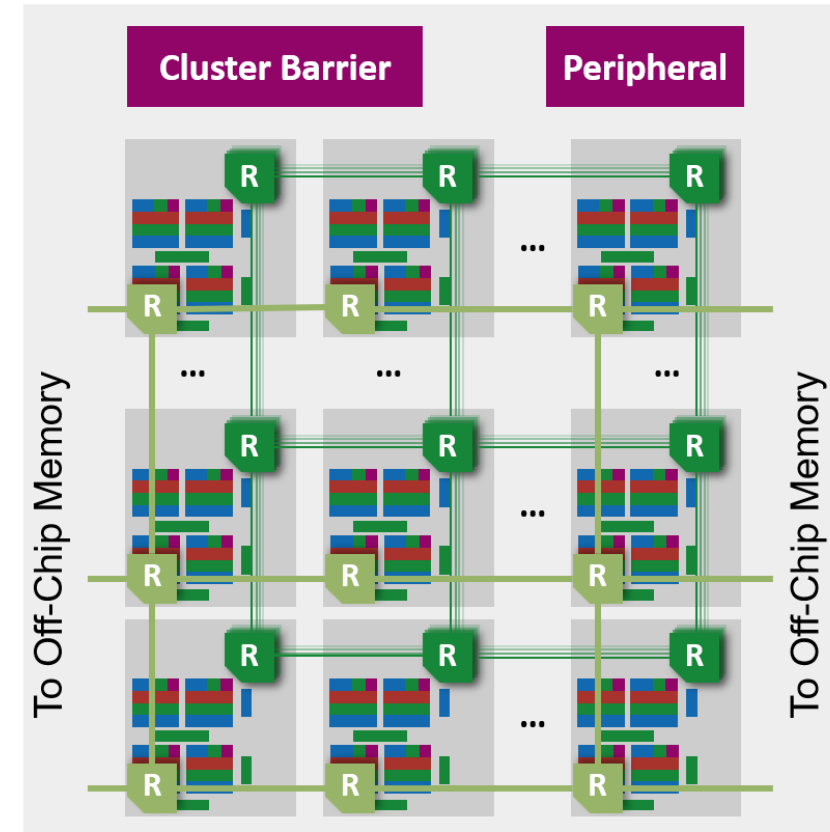
Tile



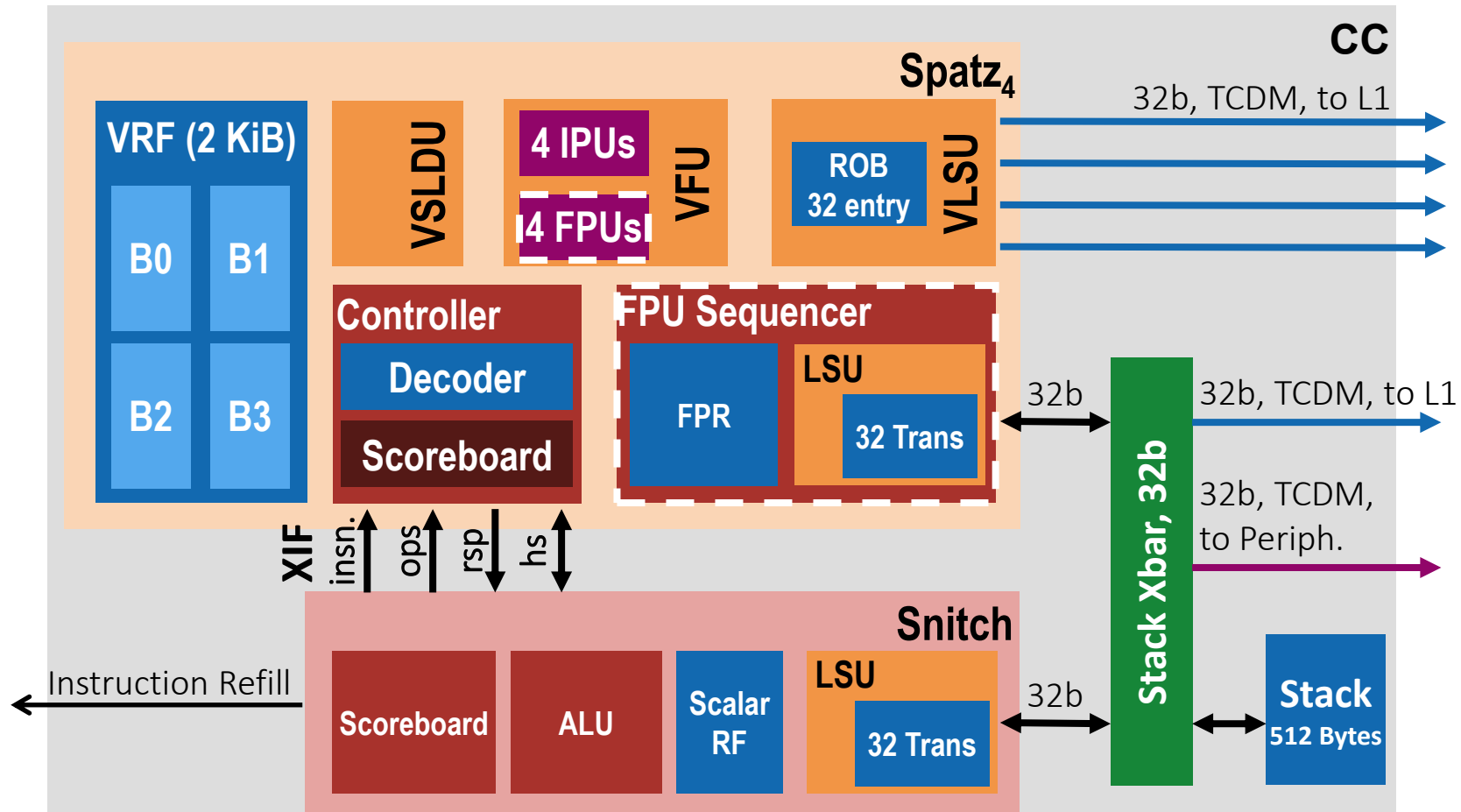
Group



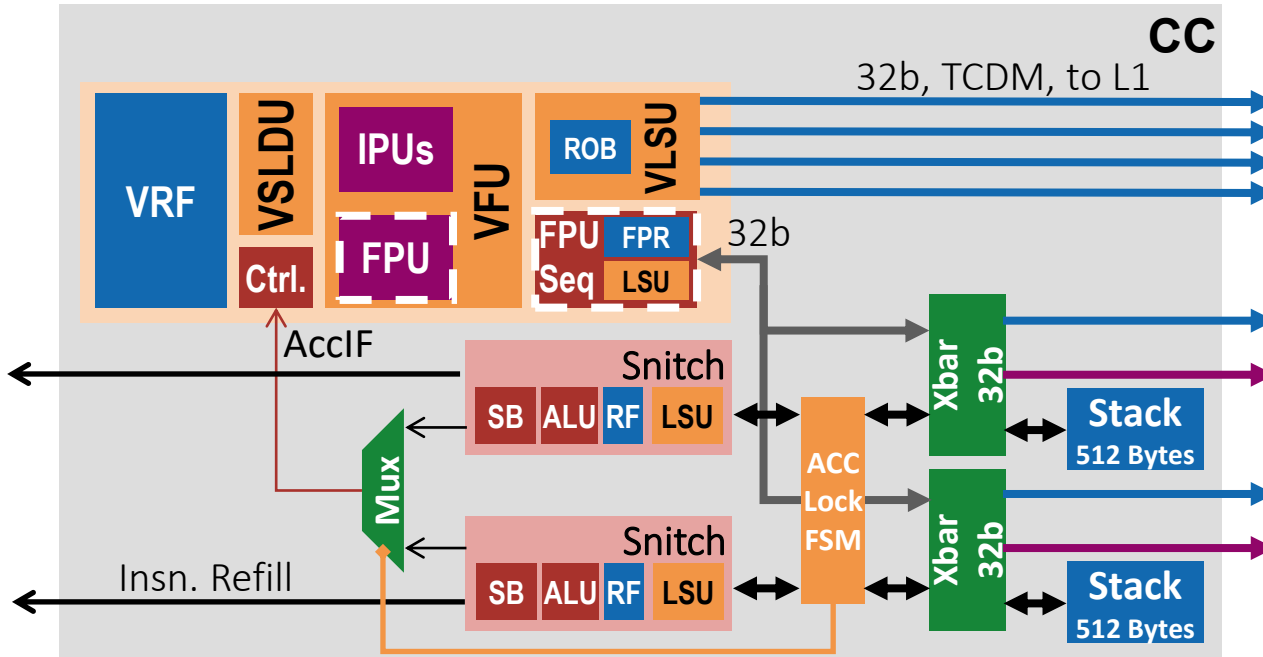
Cluster



Hardware Development (Cluster)



Hardware Development (Cluster)

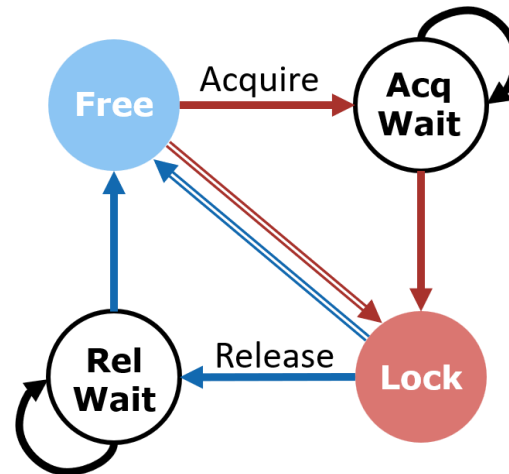


• Free mode

- Round-robin requests from both Snitch to Spatz
- Default mode on booting, so that both Snitch can clear FPR and VRF if using uniformed bootrom.

• Lock mode

- Any Snitch can lock the access by writing to a mem-mapped address.
- The write will be taken by the FSM to switch mode
- Pause acc access during switch, and switch completes when no running acc insn.
- Spatz “private” to the owner



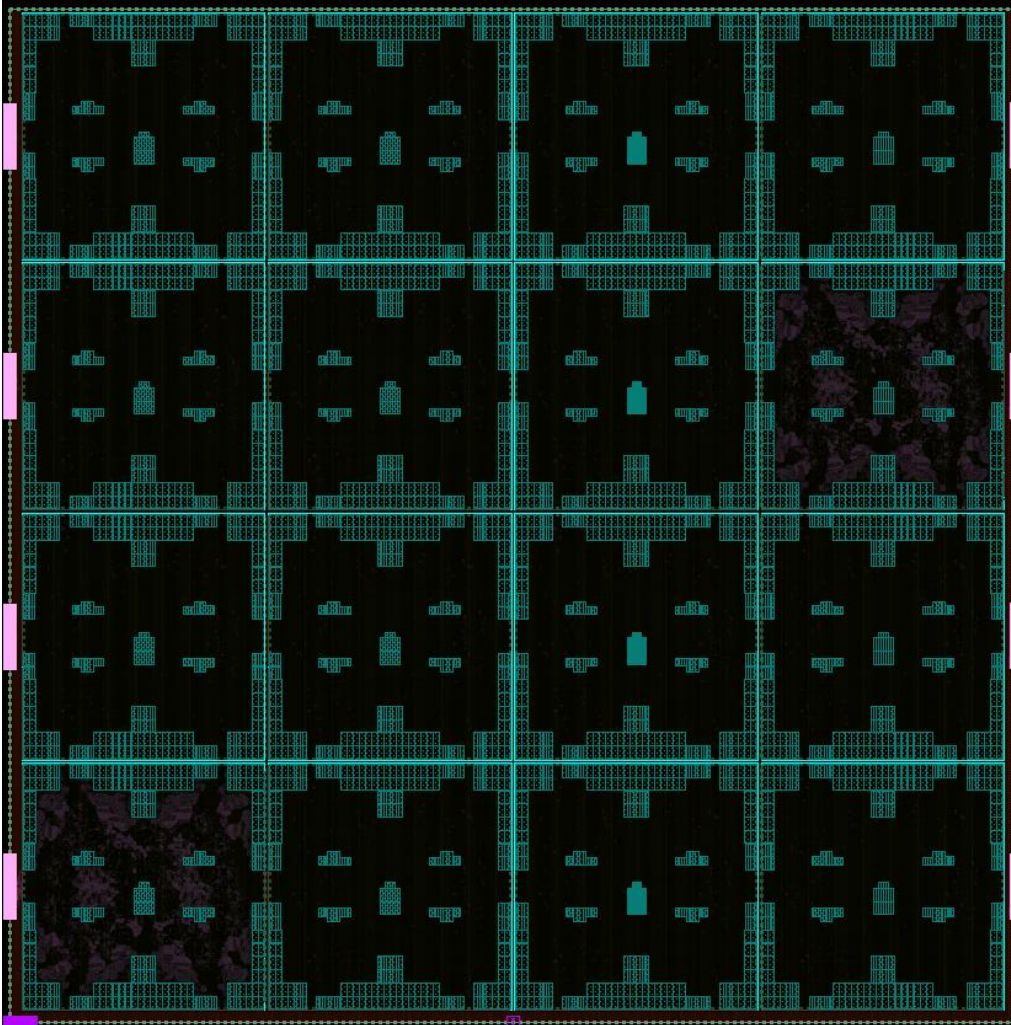
Hardware Development (Cluster)



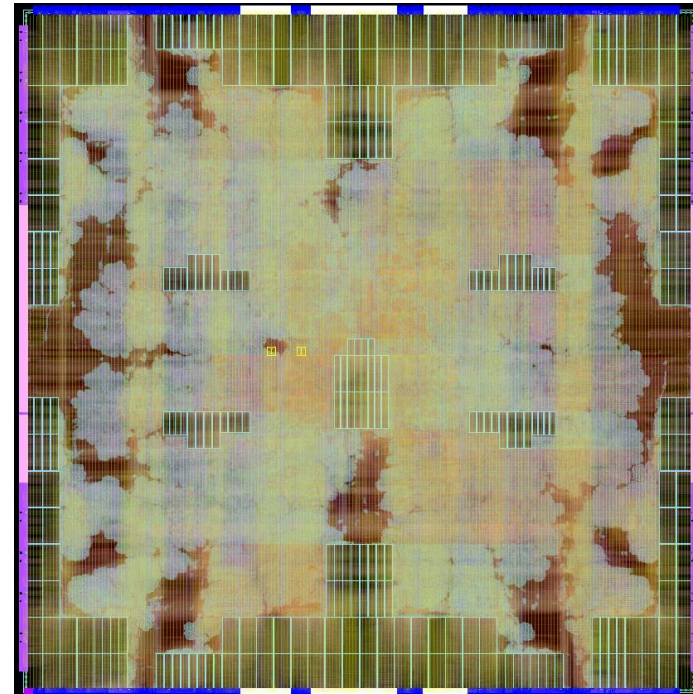
- **Developments**

- Rebase the multi-group configuration with the folded cache
 - Development branch is now merged into main
 - Functionally correct, performance checking and debugging undergoing since RTL simulation runs quite slow
- Add dual-scalar core support
 - Currently tested on 4-group configuration for faster simulation speed
 - PR opened, still needs some cleaning and testing jobs





- **Started a physical design run on the latest main (folded + multi-group)**
 - Timing: same 200ps path in cache controller (cluster-level P&R undergoing)
 - Routing: 17 shorts => Routing clean



HW dev: LR/SC bug fix



- **Reservation overwrite**

- LR from another core could overwrite a live reservation → starvation.
- **Fix:** keep current reservation + ResvTimeoutCycles (1024 cyc)

- **Incorrect SC result**

- Single outstanding-SC register could misreport a zero memory word as SC success
- **Fix:** carry is_sc/sc_fail in tcdm_user_t; avoid single-entry SC tracking.

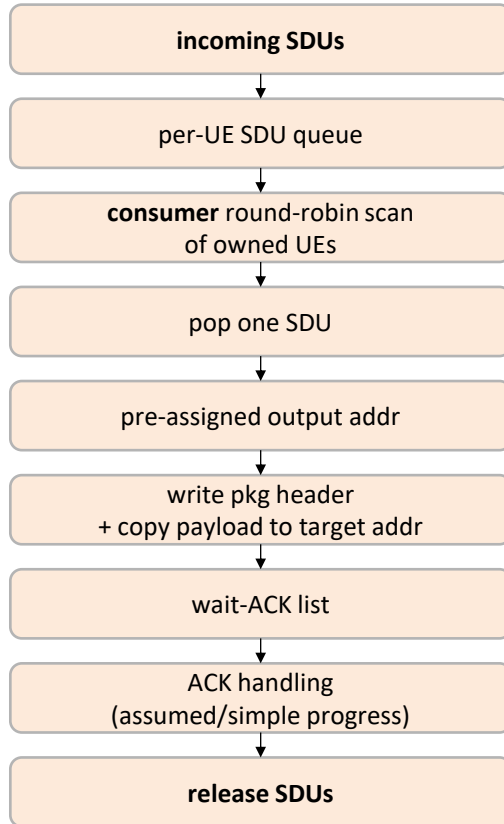
- **Validation**

- New test: lrsc-forward-progress
 - 16 cores, symmetric CAS-increment retry loop on one shared word
 - **Before:** 22/32 completed; 8 iter/core stuck
 - **After:** 256/256 completed; 16 iter/core in 25,778 cycles (~100 cycle per lr/sc pair)



SW dev: RLC Plan Stage + Processing Stage

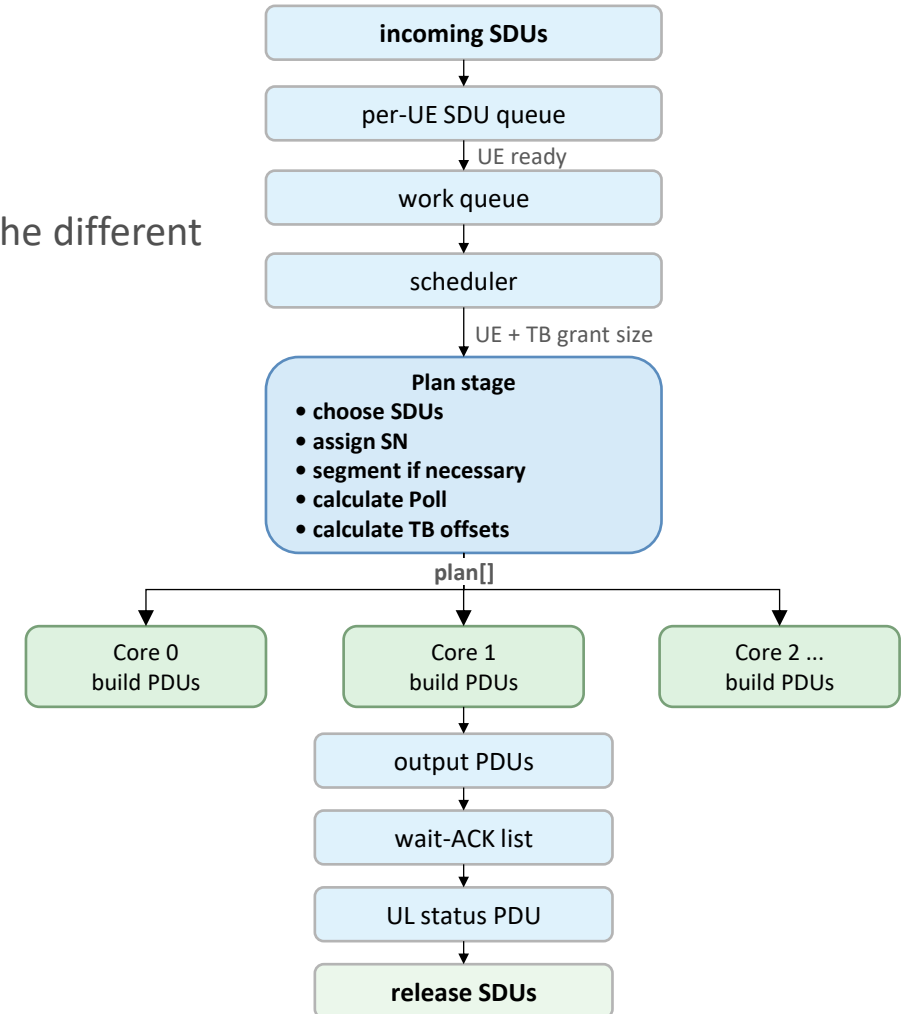
Legacy kernel (direct copy path)



Limitation:

- No resource allocation algorithm involved;
- Static task scheduling: fixed pkg producers and consumers
- No TTI control

New AM kernel (plan + parallel execute)



Main idea: Separate the different tasks:

- Pkg receiving task
- Plan task
- Pkg assembly task
- UE status task



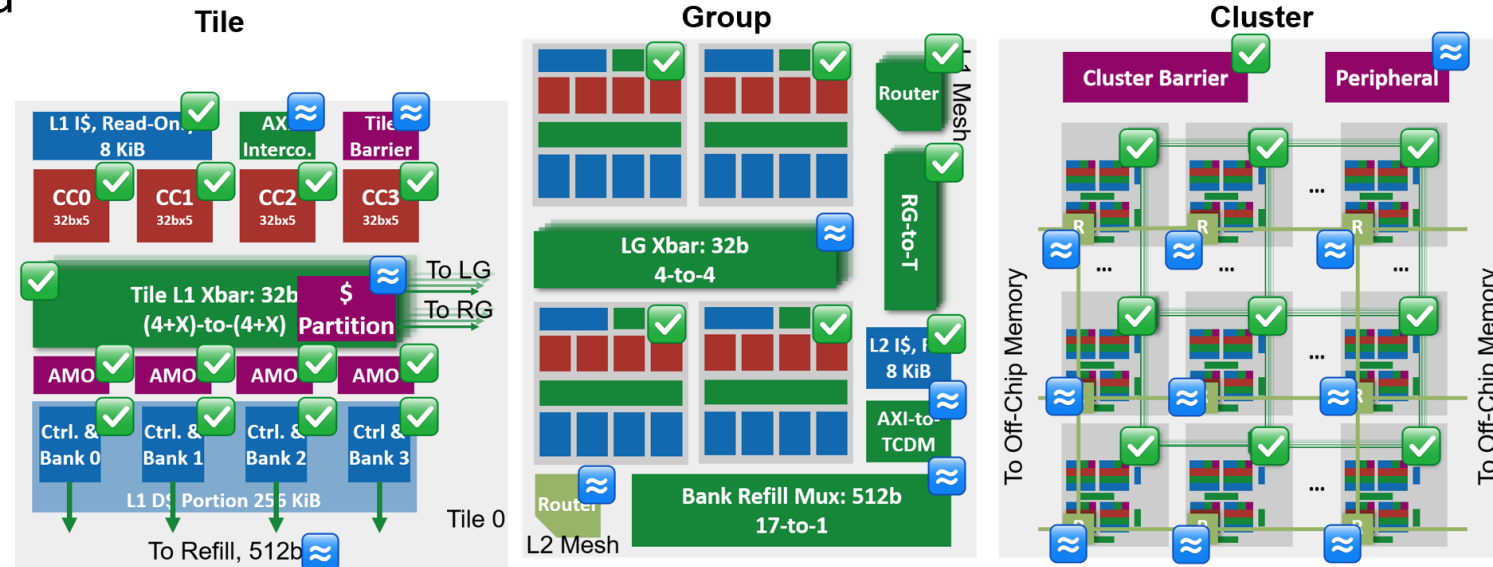
GVSoc Modeling Progress

- **L2 Refill Mesh Fixed**

- Topology now matches RTL: 6x4 mesh, 16 routers, 8 channels
- Fixed 1 KB channel interleave and 512 MiB DRAM

- **Calibration Improved**

- Better NoC latency modelling
- 64-core perf gap reduced from +60.8% → +14.8%
 - Used old RLC kernel
 - 2x2 groups x 4 tiles x 4 cores



Next Step



- **ICCD 2026**

- Unfortunately, our submission to ICCD was rejected
 - Seems like reviewer are not in this field at all...
From one reviewer: *I'm not sure its novelty, but its novelty could be a key factor in the paper's acceptance.*
 - Waiting for Luca's inputs to retarget, most likely DATE 2027
 - Also wait for his final approval to put it on axiv

- **Diyoun Side**

- Performance evaluation and debugging
- Paper adjustment and resubmission

- **Zexin Side**

- GVSoC development and calibration under multi-group config, add dual-scalar core support
- SW development based on the benchmark plan, adapt features from Johannes's kernel



Timeline



		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP1 ManyCore Architecture	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Complete													
	Interco. Optimization (multi-group) [RTL] Almost Complete													
	Timing optimization for NoC [RTL] Ongoing by colleague													
	NoC Topology Exploration [RTL, GVSoC] Ongoing by student													
WP2 Cache	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete													
	Folded Cache [GVSoC + RTL] Complete													



Timeline



	2025										2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11			
WP3 VPE	Shared vector core [RTL + GVSoC] Almost Complete																
WP4 Benchmark	Single-user kernel optimization [SW] Ongoing																
	Multi-user and more scenarios [SW] Ongoing																
Report	Final Report and Benchmark																

L1 Access Latency



- **Problem: increased L1 access latency when scaling up**
- **Solutions**
 - **(Complete, RTL & Physical) Hardware optimization**
Further optimize the interconnection and cache controller to reduce access latency on existing architecture
=> Retiming the current interconnection, remove unnecessary cuts in interco and cache
=> Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
Currently has reduced to 6 cycles:
Core => Xbar => AMO => Ctrl (2) => AMO => Core
 - **(Complete, RTL) Cache partitioning**
Partition the shared L1 to keep data localized to avoid remote access latency
 - **(Complete with some problems, RTL & Physical) WT-L1**
Explore and evaluate a private WT L1 for each core
=> Performance degrade on single tile
=> Needs more work to support larger system



L1 Access Latency



- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
 - **(Ongoing, Physical) Reduce hop latency**
Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
=> Reduce the latency by 1/2
 - **(Planned, Physical) Cut clk-domain (optional)**
If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
=> Effectively cut the latency by 1/3
 - **(Planned, GVSoC & Physical) NoC Topology**
Explore different NoC topologies
=> Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2



PE-Related



- **Problem: Improve scalar core performance**
- **Solutions**
 - **(Complete, RTL & GVSOC) Shared vector core**
Explore two Snitch sharing one Spatz configuration to improve scalar performance
Software complexity needs to be considered, especially on the possible conflicting use of register file.
Discussed Solution: critical section



RLC Workload



- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
 - **(Ongoing, SW) Kernel optimization**
Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
 - **(Planned, SW) Kernel development**
Implement the missing use cases for multi-user



Thank you!

Q&A

